

★ Constness of pointers and values

```
int num, num2;
```

```
const int const_num = 19;
```

✗ const_num = 3; ✗

```
int *ptr;
```

```
ptr = &num;
```

✗ ptr = &const_num; -

```
// int const *ptr_to_const;
```

```
const int *ptr_to_const; ] same
```

```
ptr_to_const = &const_num;
```

✗ *ptr_to_const = 3;

✓ ptr_to_const = #

✗ *ptr_to_const = 3;

✓ num = 7; printf("%d", *ptr_to_const); // 7

1. Const variable must be initialized during declaration

5000		num2	✗	int *const const_ptr;
				int *const const_ptr = #
2000	24	num		*const_ptr = 24;
			✗	const_ptr = &num2;
	1000	const_ptr2	✗	int *const const_ptr2 = &const_num;
1000	19	const_num		int const *const const_ptr_to_const = #
			✗	const_ptr_to_const = &num2;
740	2000	const_ptr	✗	*const_ptr_to_const = 17;
500	2000	ptr		
	2000	ptr_to_const		

	non-const value	const value	Is pointer const	Is value const from pointer perspective
(ptr) pointer	Yes	→ No	No	No
(ptr_to_const) pointer to const	Yes	Yes	No	Yes
constant pointer	Yes	→ No	Yes	No
constant pointer to constant	Yes	Yes	Yes	Yes

void *ptr; ✓ // we can declare pointer of type void.

void v; ✗ // we can not declare variable of type void

✓ All valid

Questions

① what will void pointer point to?

② Can we dereference it? ✗ *vptr ✗

③ what will be the scale factor? ✗

→ No arithmetic operations can be performed on void *

```
char ch;  
char *cptr = &ch;  
int i, *iptr = &i;  
*cptr = 'x';  
*iptr = 9;
```

Note: you can assign address of variable of any type to void pointer and void pointer can be assigned to any pointer type.

```
int j=5;  
int i=7, *iptr = &i;  
void *vptr;  
vptr = &i;  
vptr = iptr;  
vptr = &j;  
iptr = vptr;  
float f;  
vptr = &f, iptr = vptr;  
Student s1, *sptr;  
vptr = &s1; ✓  
sptr = vptr;
```

// Assume struct

```

#include<stdio.h>
#include<string.h>

void sort(void *ptr, int n, int elem_size, int (*compare)(void *, void *)) {
    char *cptr = (char *)ptr;
    for(int i = 0; i < n - 1; i++) {
        int min_index = i;
        for(int j = i + 1; j < n; j++) {
            if(compare(cptr + min_index * elem_size, cptr + j * elem_size)) {
                min_index = j;
            }
        }
        for(int k = 0; k < elem_size; k++) {
            char temp = cptr[min_index * elem_size + k];
            cptr[min_index * elem_size + k] = cptr[i * elem_size + k];
            cptr[i * elem_size + k] = temp;
        }
    }
}

int compare_int(void *address1, void *address2) {
    if(*(int *)address1) > *((int *)address2))
        return 1;
    return 0;
}

int compare_float(void *address1, void *address2) {
    if(*(float *)address1) > *((float *)address2))
        return 1;
    return 0;
}

typedef
struct {
    char name[20];
    int marks;
}Student;

```

```

int compare_student_marks(void *address1, void *address2) {
    if(((Student *)address1)->marks > ((Student *)address2)->marks)
        return 1;
    return 0;
}

int compare_student_name(void *address1, void *address2) {
    if(strcmp(((Student *)address1)->name, ((Student *)address2)->name) > 0)
        return 1;
    return 0;
}

int main() {

    int arr[5] = {3, 2, 1, 6, 4};
    sort(arr, 5, sizeof(int), compare_int);
    for(int i = 0; i < 5; i++)
        printf("%d ", arr[i]);
    printf("\n\n");

    float arr2[5] = {3.3, 2.2, 1.1, 6.6, 4.4};
    sort(arr2, 5, sizeof(float), compare_float);
    for(int i = 0; i < 5; i++)
        printf("%.1f ", arr2[i]);
    printf("\n\n");

    Student students[5] = {{ "Jalaj", 24}, {"Vanaj", 23}, {"Khush", 32},
        {"Ditya", 35}, {"Manav", 36}};
    sort(students, 5, sizeof(Student), compare_student_marks);
    for(int i = 0; i < 5; i++)
        printf("%s %d\n", students[i].name, students[i].marks);
    printf("\n");

    sort(students, 5, sizeof(Student), compare_student_name);
    for(int i = 0; i < 5; i++)
        printf("%s %d\n", students[i].name, students[i].marks);
    printf("\n");

    return 0;
}

```